

# Hora de tentar sozinhos

Vocês já conhecem o dataset mpg do desafio de Regressão — agora é a vez de aplicar Regressão Logística nele, sozinhos (ou em duplas), sem competição.

Mesma base, problema diferente: em vez de prever o consumo exato, vamos prever se um carro é 'econômico' ou não.

**O objetivo é praticar o pipeline inteiro, do começo ao fim, sem ajuda passo a passo.**



# Importações necessárias

Comecem importando tudo que vão precisar — mesmas bibliotecas que já usamos na teoria.

## LEMBRETE DE SINTAXE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```



## Criar a categoria

Transformem mpg (número) numa categoria binária: economico = 1 se mpg  $\geq$  23, senão 0.

### LEMBRETE DE SINTAXE

```
df = pd.read_csv("mpg_desafio_alunos.csv")  
df["economico"] = (df["mpg"] >= 23).astype(int)
```

## Escolher a feature

Olhem a correlação de cada variável numérica com economico, e escolham 1 feature.

### LEMBRETE DE SINTAXE

```
df.corr(numeric_only=True)["economico"]  
  
X = df[["weight"]] # troquem pela feature escolhida  
y = df["economico"]
```

## Tratar valores nulos

Se escolherem horsepower, lembrem de tratar os valores faltando antes de continuar.

### LEMBRETE DE SINTAXE

```
df = df.dropna(subset=["horsepower"])
```



## Separar treino e teste

Use `stratify=y` desde já — já sabemos por que isso importa em classificação.

### LEMBRETE DE SINTAXE

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=5, stratify=y  
)
```



## Treinar a regressão logística

Mesma sintaxe de sempre: fit() nos dados de treino.

### LEMBRETE DE SINTAXE

```
modelo = LogisticRegression()  
modelo.fit(X_train, y_train)
```

## Visualizar a sigmoide ajustada

Vejam a curva que o modelo aprendeu, em cima da feature que vocês escolheram.

### LEMBRETE DE SINTAXE

```
feature_nome = X.columns[0]
x_linha = np.linspace(df[feature_nome].min(), df[feature_nome].max(), 200).reshape(-1, 1)
prob_linha = modelo.predict_proba(x_linha)[: , 1]

plt.scatter(df[feature_nome], df["economico"], alpha=0.6)
plt.plot(x_linha, prob_linha, color="green", linewidth=3)
plt.axhline(0.5, color="red", linestyle=":")
plt.xlabel(feature_nome)
plt.ylabel("Probabilidade de ser economico")
plt.show()
```





## Avaliar com acurácia

Um primeiro número, rápido — sem aprofundar ainda (veremos em breve).

### LEMBRETE DE SINTAXE

```
pred = modelo.predict(X_test)
print(f"Acuracia: {accuracy_score(y_test, pred):.2%}")
```



## Avaliação final

Rodem no mpg\_desafio\_final.csv, com a mesma coluna economico calculada.

### LEMBRETE DE SINTAXE

```
df_final = pd.read_csv("mpg_desafio_final.csv")
df_final["economico"] = (df_final["mpg"] >= 23).astype(int)

X_final = df_final[["weight"]]
y_final = df_final["economico"]

pred_final = modelo.predict(X_final)
print(f"Acuracia final: {accuracy_score(y_final, pred_final):.2%}")
```



## Refletir

Comparando com o desafio de Regressão: o que mudou na prática, de verdade, entre prever um número (mpg) e prever uma categoria (econômico)? O que continuou igual?

**A sintaxe do Scikit-Learn mal muda — o que muda é como pensamos o problema e como avaliamos o resultado.**



# Pipeline completo, do zero, sozinhos

Categoria nova, feature escolhida com dado, treino/teste com stratify, modelo treinado e visualizado, avaliado duas vezes — tudo sem roteiro passo a passo.

**A seguir: KNN — decidir pela vizinhança.**